ACTIVITY – 10

Travelling Salesperson Problem Using Prolog

Submitted by

D. Jeshurun Silas

Registration Number: 192525324

Programming Language: Prolog
Software Used: SWI-Prolog
Activity: Find the shortest round trip through five cities

1. Aim of the Activity

To write and execute a Prolog program that examines possible routes between five cities and finds a shortest tour. The tour must start at city A, visit every other city exactly once, and return to city A.

2. Problem Statement

A map contains five cities—A, B, C, D and E. Some cities are connected by roads, and each road has a distance. We must find a route that starts at A, visits B, C, D and E exactly once each, and comes back to A. The route with the smallest total distance is the answer.

3. Understanding the Given Map

Each line below represents a road. The number beside the road is its distance. For example, road(A,B,4) means the road between A and B has distance 4.

City 1	City 2	Distance
A	B	4
A	C	1
A	E	3
B	D	5
B	E	2
C	D	3
C	E	2
D	E	1

The roads are treated as two-way roads. Therefore, if A is connected to B, travel between A and B is possible in either direction at the same distance.

4. What Is the Travelling Salesperson Problem?

The Travelling Salesperson Problem (TSP) is a route-planning problem. Imagine a person who must visit several cities and return home. The person wants to travel the least possible distance. In this activity, the cities are A, B, C, D and E, and A is the starting and ending city.

A valid tour cannot skip a city or visit a city twice before returning to A. The final return to A is required, but A is not counted as a second visit to a new city.

5. Prolog Program

The following is a complete DFS/backtracking-style Prolog solution. It explores possible next cities, avoids cities already visited, adds the distance of each road, and compares completed tours.

```
% Roads and distances in the map
road(a, b, 4).
road(a, c, 1).
road(a, e, 3).
road(b, d, 5).
road(b, e, 2).
road(c, d, 3).
road(c, e, 2).
road(d, e, 1).

% Roads can be travelled in either direction
distance(X, Y, D) :- road(X, Y, D).
distance(X, Y, D) :- road(Y, X, D).

% Find the shortest tour starting from Start
tsp(Start, BestTour, BestDistance) :-
    findall(Distance-Tour,
            complete_tour(Start, Distance, Tour),
            Tours),
    sort(Tours, [BestDistance-BestTour | _]).

% Generate one complete tour
complete_tour(Start, Distance, Tour) :-
    dfs(Start, Start, [Start], ReversePath, 0, Distance),
    reverse(ReversePath, Path),
    append(Path, [Start], Tour).
```

```
% Stop when all five cities have been visited
dfs(Current, Start, Visited, Visited, Cost, Total) :-
    length(Visited, 5),
    distance(Current, Start, ReturnDistance),
    Total is Cost + ReturnDistance.

% Otherwise, choose a connected city not visited yet
dfs(Current, Start, Visited, FinalVisited, Cost, Total) :-
    length(Visited, Count),
    Count < 5,
    distance(Current, Next, RoadDistance),
    \+ member(Next, Visited),
    NewCost is Cost + RoadDistance,
    dfs(Next, Start, [Next | Visited], FinalVisited,
        NewCost, Total).
```

6. Explanation of the Code in Simple Words

- **Road facts:** The lines beginning with road(...) store the roads and their distances. For example, road(a,b,4) records a road from A to B with distance 4.
- **Two-way travel:** The distance rules allow the program to use a road in either direction. A road from A to B also allows travel from B to A.
- **tsp/3:** The main instruction asks Prolog to collect all complete tours and sort them by distance. The first item after sorting is the shortest tour.
- **complete_tour/3:** This builds a full route. It starts at the chosen city, uses DFS to visit all five cities, then adds the starting city to the end.
- **dfs/6:** DFS means Depth-First Search. The program follows one possible route as far as it can before trying another route. Prolog backtracking lets it explore other choices.
- **member/2 and \\+:** member(Next, Visited) checks whether a city is already in the visited list. The operator \\+ means 'not provable', so the program only chooses a city that has not already been visited.
- **Cost calculation:** NewCost adds the distance of the next road to the distance already travelled. When all five cities have been visited, the program adds the road back to the starting city.

- **sort/2:** The completed tours are sorted by their numeric distances. The shortest distance and its tour become the answer.

7. Steps to Run the Program in SWI-Prolog

1. Open Notepad or VS Code and paste the complete program shown in Section 5.
2. Save the file as tsp.pl. If using Notepad, choose 'Save as type: All files' so that the filename does not become tsp.pl.txt.
3. Open the SWI-Prolog application.
4. At the ?- prompt, load the file by entering: consult('D:/CSA1718/Activities/tsp.pl'). Use the path where you actually saved the file.
5. Press Enter. The response true. means Prolog loaded the file successfully.
6. At the next ?- prompt, enter: tsp(a, Tour, Distance).
7. Press Enter to see the shortest tour and its total distance.

8. Output Screenshot

The screenshot below is the actual SWI-Prolog console output supplied for this activity. It shows that the file was loaded successfully and that the query returned a tour with distance 11.

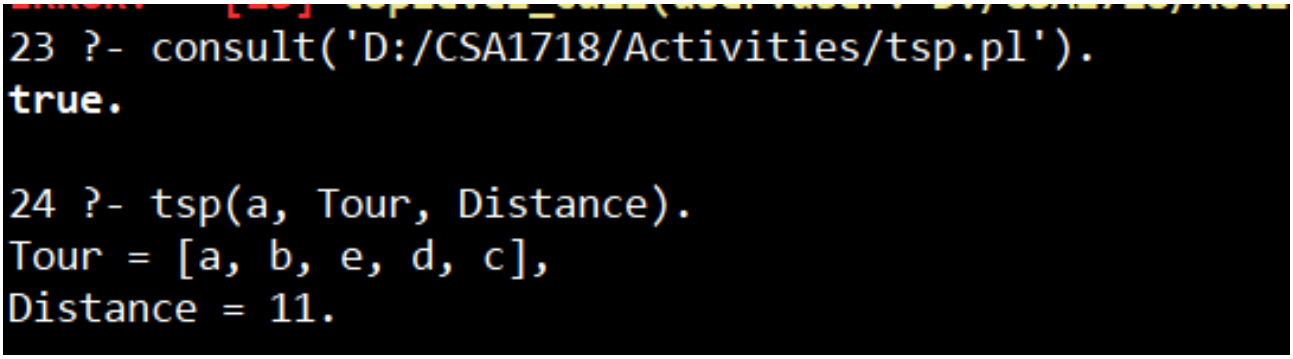


Figure 1. SWI-Prolog output for tsp(a, Tour, Distance).

9. Result and Distance Calculation

The program output is:

Tour = [a, b, e, d, c],

Distance = 11.

The Tour list shows the cities before the final return to A. The complete trip is A → B → E → D → C → A.

Distance calculation:

- A → B = 4
- B → E = 2
- E → D = 1
- D → C = 3
- C → A = 1
- **Total = 4 + 2 + 1 + 3 + 1 = 11**

10. Conclusion

In this activity, a weighted road map was represented using Prolog facts. The program used depth-first exploration and backtracking to generate valid tours, calculated the total distance of each tour, and selected a shortest one. The resulting tour is A → B → E → D → C → A, with a total distance of 11. This activity demonstrates how Prolog can represent facts and search for solutions to a route-planning problem.

11. Key Terms

- **City/Node:** A location in the map, represented by a, b, c, d or e.
- **Road/Edge:** A connection between two cities.
- **Weight/Distance:** The number written beside a road; it represents the travel cost.
- **Tour:** A route that visits every city once and returns to the starting city.
- **DFS:** Depth-First Search; it follows one route deeply before trying alternatives.
- **Backtracking:** When one choice does not finish the task, Prolog returns and tries another choice.